

4 Sécurité du déploiement

Mesures de sécurité pour le déploiement de **Hublot** (DIRM Méditerranée).

Annexe associée : [Annexe 07](#)

1. HTTPS

Canal	Mise en œuvre
Netlify (PROD)	Certificat TLS géré automatiquement par Netlify
NAS (local)	HTTP par défaut ; HTTPS possible via Reverse Proxy DSM + certificat interne

Vérification :

```
curl -I https://dirmhublot.netlify.app
```

Attendu : URL en `https://` , réponse `200` ou `304` .

2. Variables d'environnement sécurisées

Les secrets ne sont **jamais** dans le dépôt Git.

Variable (exemples)	Où la configurer	Usage
<code>VITE_APP_USERNAME</code>	Netlify → Environment variables	Connexion application
<code>VITE_APP_PASSWORD</code>	Netlify → Environment variables	Connexion application
<code>NETLIFY_DATABASE_URL</code>	Netlify (extension Neon)	Accès base PostgreSQL

En local : fichier `.env` (listé dans `.gitignore`).

3. Gestion des secrets

Règle	Application
Pas de commit de secrets	<code>.gitignore</code> : <code>.env</code> , <code>agents.json</code> , <code>*.xlsx</code> , certificats
Séparation des rôles	Comptes Netlify / GitHub distincts des comptes métier
Données RH internes	Hors dépôt public ; accès restreint aux équipes habilitées
Rotation	Changer les mots de passe en cas de fuite suspectée

Fichier de référence : [Annexe 04 — .gitignore](#)

4. Pas de données sensibles exposées

- Aucun export métier dans le README public.
- Données agents non versionnées (`src/data/` , `trdata/` ignorés).
- API / base en production protégées par authentification applicative.

5. Headers de sécurité

Configurés dans `netlify.toml` (Annexe 02) :

Header	Valeur	Protection
<code>X-Frame-Options</code>	<code>DENY</code>	Clickjacking
<code>X-Content-Type-Options</code>	<code>nosniff</code>	MIME sniffing
<code>Referrer-Policy</code>	<code>strict-origin-when-cross-origin</code>	Fuite de référent
<code>Permissions-Policy</code>	<code>camera/micro/geo</code> désactivés	Permissions navigateur

Vérification :

```
curl -I https://dirmhublot.netlify.app | grep -i x-frame
```

Configuration Nginx (NAS) : voir `nginx.conf` (CSP, X-Frame-Options en commentaire ou actif selon version).

6. Tests de vulnérabilité (npm audit)

Contexte	Commande	CI
Local	<code>npm audit</code>	—
Pipeline	<code>npm run audit:prod</code>	Bloquant en CI (critical + high hors allowlist) — voir [SECURITE_AUDIT.md](./1.2 Préparer le déploiement d'une application/1.2.4.3 Audit des dépendances.md)

Interprétation : certaines alertes concernent des dépendances de **développement** (Vitest, outils de build). Le site en production sert uniquement les fichiers statiques du dossier `build/`.

Plan d'action :

1. `npm audit` régulier
2. `npm audit fix` pour les correctifs sans breaking change
3. Documenter les vulnérabilités résiduelles acceptées

7. Authentification applicative

- Page de connexion avant accès au tableau de bord.
- Identifiants injectés au build via variables d'environnement (`import.meta.env`).
- Session côté navigateur (`sessionStorage`) — à renforcer (JWT / SSO) en évolution future.

Test manuel : voir [PLAN_TEST.md](./1.3 Rédiger des scripts dans la démarche DevOps/1.3.7 Automatiser les tests en DevOps.md) — scénario authentification.

8. Checklist avant mise en production

Utiliser [CHECKLIST_SECURITE.md](./1.2 Préparer le déploiement d'une application/1.2.4.4 Checklist sécurité.md) :

- ☐ Mots de passe forts configurés sur Netlify
- ☐ `.env` absent du dépôt
- ☐ CI verte sur `main`
- ☐ Headers vérifiés en production
- ☐ Rollback testé (au moins une fois)

Compléments : [SECURITE.md](./1.2 Préparer le déploiement d'une application/1.2.4.1 Mesures de sécurité applicative.md), [DEPLOIEMENT_SECURISE.md](./1.3 Rédiger des scripts dans la démarche DevOps/1.3.1 Les bases du déploiement automatique.md)

Synthèse jury

« Le déploiement est sécurisé par HTTPS en production, des variables d'environnement pour les secrets, l'exclusion des données RH du dépôt, des headers HTTP durcis et un audit npm intégré à la CI. L'authentification protège l'accès aux tableaux de bord. »